

System Programming

Assignment #1 : decimal to binary calculator

Goal

본 과제에서는 **10진수 정수를 입력받아, 2진수 문자열로 출력하는 어셈블리 프로그램**을 만들어보는 것을 목표로 합니다.

Requirements

본 과제에 제출할 프로그램은 다음 요구사항을 따라야합니다.

- **입출력 처리**

모든 입출력 처리는 디바이스를 통해서 진행합니다.

- 입력은 0x03 디바이스로부터 받습니다.
- 출력은 0x04 디바이스로 수행합니다.

- **입력 및 출력 사양**

- 입력 범위 (Input Range): $0 \leq N \leq 16,777,215$ (즉, $0 \sim 2^{24} - 1$)
- 출력 양식: 입력받은 정수 데이터를 1 바이트 단위로 출력합니다. 바이트간 여백을 두어야합니다.

- **입출력 예시**

입력 (Input device : 03.dev)	출력 (output device : 04.dev)	검증 포인트
1	00000000 00000000 00000001	기본 3바이트 구조 확인
256	00000000 00000001 00000000	바이트 경계 2^8 처리 확인
65536	00000001 00000000 00000000	2^{16} 자리 올림 확인 (양수 범위 내)

Note.

SicTools에서는 입출력 인터페이스를 제공하기 위해 사전 설정된 디바이스를 사용합니다. 예를 들어, 표준 입출력인 `STDIN`, `STDOUT`, `STDERR` 는 각 `0x00`, `0x01`, `0x02` 로

mapping 되어 있습니다. 디바이스 `0x03` ~ `0xFF` 의 경우, 실행 경로에 있는 파일로부터 입출력을 진행합니다.

참조: <https://jurem.github.io/SicTools/documentation/simulator> - (Input / Output)

Instruction

다음 과정으로 과제를 진행할 것을 **권장**합니다.

STEP 1. Setup devices

입출력을 위한 디바이스 파일을 설정합니다.

1. .asm 소스 파일이 위치한 경로에 `03.dev` 파일을 생성합니다. 이 파일로부터 프로그램이 입력을 받습니다.
2. 파일 안에 변환하고자 하는 10진수 숫자를 입력합니다.
[ex. (`03.dev` 내용) : 10]
3. 동일한 경로에 빈 파일인 `04.dev` 를 생성합니다. 프로그램이 출력하는 내용은 해당 파일에 쓰여집니다.

• 장치 파일 위치 및 프로그램 실행 예시

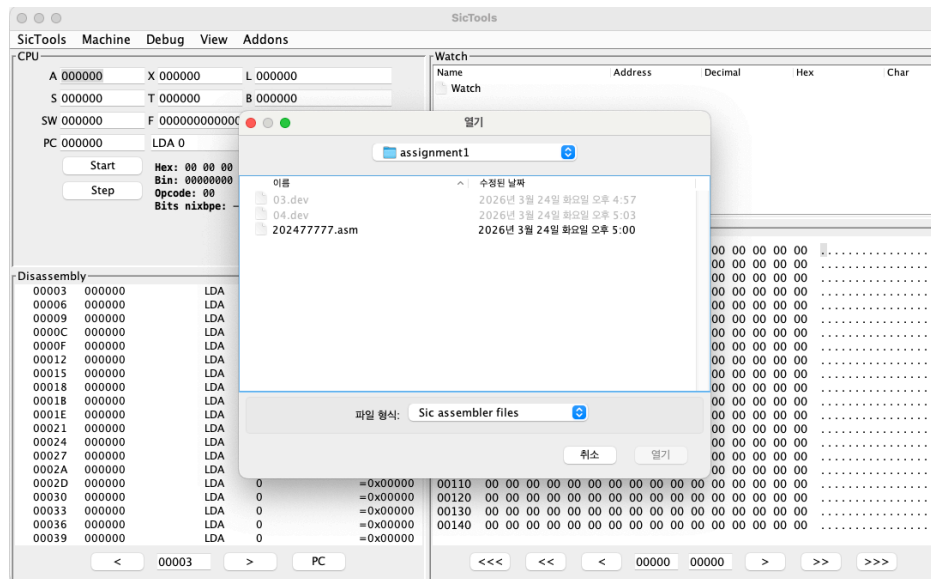
(`03.dev` , `04.dev` , 그리고 본인의 `.asm` 파일은 반드시 **동일한 폴더** 내에 존재해야 합니다.)

```
SicTools/
├─ assignment1/          <-- 본인의 프로젝트 폴더
│   ├── 03.dev           (입력 파일)
│   ├── 04.dev           (출력 파일)
│   └─ 202477777.asm     (소스 코드)
```

이제 프로젝트 폴더(assignment1) 내에서 SicTools를 실행하면, 디바이스를 정상적으로 인식합니다. 다음의 커맨드를 참고하세요.

```
cd ~/Desktop/SicTools/assignment1
java -jar ../out/make/sictools.jar
```

이후 아래와 같이 asm 파일을 로드합니다.



STEP 2. Read from DEV '0x03', and write to DEV '0x04'

먼저 디바이스 파일이 잘 셋업되었는지 확인합니다.

1. 디바이스 0x03 으로부터 1바이트를 읽어 0x04로 단순 출력해봅니다.
(SicTools 101의 I/O Part를 참고)
2. **04.dev** 파일을 확인하여 출력이 제대로 진행되었는지 확인합니다.

STEP 3. Write program

이제 본 과제의 목표에 따라, 읽은 값을 2진수로 출력하는 프로그램을 작성합니다.

1. 디바이스 파일로부터 1 바이트씩 데이터를 읽어 정수로 변환하는 코드를 작성합니다.
2. 올바르게 읽은 정수를 2진수로 변환하는 알고리즘을 작성합니다.
3. 이제 해당 2진수를 양식에 맞춰 출력하는 코드를 작성합니다.

Submission

본 과제 제출은 다음 기준을 따릅니다.

• 제출 기한

- 1차 : 03/25 ~ 04/08 11:59 PM
- 2차 : 04/13 ~ 04/19 11:59 PM

(1차 기한 이후 재제출 시 점수는 70% 인정됩니다.)

• 제출 파일

- SicTools에서 실행 가능한 `.asm` 파일의 이름을 본인의 학번으로 바꾸어 제출합니다.
(ex. 202477777.asm)

Grading Policy

본 과제는 총 10점 만점으로, 제출한 프로그램이 테스트케이스를 수행한 결과에 따라 채점합니다. 단, 테스트케이스는 공개하지 않습니다.

기타 감점사항

- 제출 형식이 잘못된 경우, 예외없이 -1점
- 지각제출 시 예외없이 기본 -1점, 시간당 -0.5점

Appendix (Hint)

1. 데이터의 내부 표현 및 변환 개념

컴퓨터 시스템(SIC/XE 시뮬레이터 포함) 내에서 모든 수치 데이터는 물리적인 전압 상태를 반영하는 **비트(Bit)** 단위로 관리됩니다.

- **이진 인코딩** : 사용자가 입력한 10진수 정수는 레지스터나 메모리에 저장될 때 0과 1의 조합인 **이진 비트열** 형태로 인코딩되어 저장됩니다.
- **추상화와 실제** : 프로그래밍 언어 수준에서는 10진수로 보일지라도, SIC/XE 머신의 24비트 **워드(Word)** 내부에서는 하드웨어가 이해할 수 있는 이진 데이터로 존재하게 됩니다. 본 과제는 이 내부의 비트 상태를 다시 사용자가 읽을 수 있는 문자열로 "디코딩"하는 과정입니다.

2. 10진수 - 2진수 변환 방법론

- **반복적 제법 (반복 나누기)**
수학적으로 가장 널리 알려진 방식으로, 10진수 정수를 2진수의 기수(Base)인 2로 반복해서 나누는 과정입니다.
- **비트 마스크 (비트 추출)**
컴퓨터 내부 레지스터에 이미 2진수로 저장된 값을 논리 연산을 통해 읽어내는 방식입니다.
- **시프트 연산 (위치 이동)**
데이터의 비트들을 왼쪽이나 오른쪽으로 밀어서 자릿수를 이동시키는 방식입니다.

주로 비트 마스킹과 결합하여 전체 비트를 순차적으로 확인할 때 사용합니다.

- **참고자료**

1. <https://www.tcpschool.com/codingmath/notation> 진법 변환 참고
2. <https://www.geeksforgeeks.org/dsa/binary-representation-of-a-given-number/>

비트 마스킹 & 시프트 연산을 통한 변환 + 반복적 제법 변환

3. ASCII 문자로의 변환

- **출력 원리** : 컴퓨터 메모리에 있는 숫자 0이나 1은 데이터일 뿐이므로, 화면에 표시하기 위해서는 해당 숫자를 **ASCII 코드 문자**로 변환해야 합니다.
- **변환 로직** : 숫자 0 또는 1에 16진수 0x30 (10진수 48)을 더하면 ASCII 문자인 '0'(0x30) 또는 '1'(0x31)이 되어 출력 장치를 통해 화면에 표시됩니다.

4. A 레지스터 초기화

- **LDCH** 사용 전 **CLEAR A** 명령어를 입력하여 기존 값을 제거하는 걸 추천합니다. 해당 변수나 메모리 공간을 0으로 초기화하지 않고 사용할 경우 남아있던 기존 값에 의해 새롭게 입힌 데이터가 오염될 수 있습니다.

5. 단계별 디버깅

- SicTools의 **Step** 실행 기능을 사용하여 레지스터와 메모리 값이 의도대로 변하는지 확인하며 개발하세요.

6. 16진수 및 2진수 문자 변환 (ASCII)

- 숫자를 화면에 출력하려면 해당 숫자의 **ASCII 코드** 값으로 변환해야 합니다.
- 숫자가 0에서 9 사이일 때는 **48(16진수로 0x30)**을 더합니다.

ex. 숫자 0 + 48 = 48 → ASCII 문자 '0'

- 숫자가 10(0xA)에서 15(0xF) 사이일 때는 **55(16진수로 0x37)**를 더합니다.

ex. 숫자 12 (0xC) + 55 = 67 → ASCII 문자 'C'

5. 프로그램 종료

- 프로그램의 마지막에는 **HALT J HALT** 와 같은 무한 루프 구조를 사용하여 실행 흐름이 데이터 영역으로 넘어가지 않도록 방지해야 합니다.